Chapter 3: Java Data Structures Concepts and Coding to an Interface

Chapter Objectives

1. Differentiate the terms Abstract Class, Concrete Class and Interface
2. Know the difference between overriding and Overloading
3. Learn the Java Generic class. Focus in Collections, Lists, Sets, Map
4. Learn how to code to an Interface

Introduction

This section introduces some basic data structures. A Datastructure is a set of types, a set of functions, and a set of axioms, implying that a datastructure is a type with implementation. In our object-oriented programming, type with implementation means concrete class.

The datastructure is a class definition is too broad because it embraces Employee, Vehicle, Account, and many other real-world entity-specific classes as datastructures. Although those classes structure various data items, they do so to describe real-world entities (in the form of objects) instead of describing container objects for other entity (and possibly container) objects.

This containment idea leads to a more appropriate datastructure definition: a datastructure is a container class that provides storage for data items, and capabilities for storing and retrieving data items. Examples of container datastructures: arrays, linked lists, stacks, and queues.

In this course, we will focus on a few inbuilt data structures like Collection, List, Set and Map. These are datastructures are are crucial for navigating through this course.

First of all, we will start with the recap of Java's OO concepts

Interface , Abstract, Concrete Classes and Annotations

Interface

A Java interface is a class, which only declare abstract methods and variables. An abstract methods is a method without an mplementation body. Interfaces are a way to achieve polymorphism in Java.

Below is an example Interface

```
 public interface MyInterface {

    public String hello = "Hello World";

    public void sayHello();
}
```

An interface is declared using the Java keyword interface. Just like with classes, an interface can be declared public or package scope (no access modifier).

The interface contains one variable and one method. The variable can be accessed directly from the interface, like this:

```
MyInterface.hello;
```

The method, however, needs to be implemented by some class, before you can access it as show below

```
public class MyInterfaceImpl implements MyInterface {

    public void sayHello() {
        System.out.println(MyInterface.hello);
    }
}
```

And then the usage

```

MyInterface myInterface = new MyInterfaceImpl();

myInterface.sayHello();
```

Note that A class can implement multiple interfaces. In that case the class must implement all the methods declared in all the interfaces implemented

Abstract

Abstract classes in Java are classes, like Interfaces, which cannot be instantiated. The purpose of an abstract class is to function as a base for subclasses.

You declare that a class is abstract by adding the abstract keyword to the class declaration.

```
public abstract class MyAbstractClass {

}
```

The difference between an Interface and an abstract class is that while an interface has only abstract methods, and abstract class can have concrete methods.

An abstract class can have abstract methods. You declare a method abstract by adding the abstract keyword in front of the method declaration.

```
public abstract class MyAbstractClass {

    public abstract void abstractMethod();
}
```

An abstract method has no implementation. It just has a method signature.

Below is a sample abstract class with two concrete methods and one abstract method

```
public abstract class MyAbstractProcess {

    public void process() {
        stepBefore();
        action();
        stepAfter();
    }

    public void stepBefore() {
        //implementation directly in abstract superclass
    }

    public abstract void action(); // implemented by subclasses

    public void stepAfter() {
        //implementation directly in abstract superclass
    }
}
```

Concrete Class

A Java concrete class is a template for how objects of that class looks. A concrete class is used to create Java objects.

To create objects of a certain class, you use the new keyword as shown in the diagram below where three car objects are created from the concrete Car class

```
public class Car {

}

Car car1 = new Car();
Car car2 = new Car();
Car car3 = new Car();
```

Annotations

Java annotations are used to provide meta data for to Java code. Being meta data, the annotations do not directly

affect the execution of your code, although some types of annotations can actually be used for that purpose.

Java annotations were added to Java from Java 5.

Java annotations are typically used for the following purposes:

1. Compiler instructions
2. Build-time instructions
3. Runtime instructions

An annotation in its shortest form looks like this:

```
@Test
```

The @ character signals to the compiler that this is an annotation. The name following the @ character is the name of the annotation. In the example above the annotation name is Test

You can put Java annotations above classes, interfaces, methods, method parameters, fields and local variables. Here is an example with anotations

```
@Entity
public class Vehicle {

    @Persistent
    protected String vehicleName = null;


    @Getter
    public String getVehicleName() {
        return this.vehicleName;
    }

    public void setVehicleName(@Optional vehicleName) {
        this.vehicleName = vehicleName;
    }

    public List addVehicleNameToList(List names) {

        @Optional
        List localNames = names;

        if(localNames == null) {
            localNames = new ArrayList();
        }
        localNames.add(getVehicleName());

        return localNames;
    }

}
```

Java comes with three annotations which are used to give the Java compiler instructions. These annotations are:

- @Deprecated
- @Override
- @SuppressWarnings

Collections, List, Set and Map

Introduction

The Java Collections API's provide Java developers with a set of classes and interfaces that makes it easier to handle collections of objects.

Rather than having to write your own collection classes, Java provides these ready-to-use collection classes for you.

The purpose of this tutorial is to give you an overview of the Java Collection classes. Thus it will not describe each and every little detail of the Java Collection classes. But, once you have an overview of what is there, it is much easier to read the rest in the JavaDoc's afterwards.

Most of the Java collections are located in the java.util package. In this course, we will just focus on Four collections API, namely Collection, List, Set and Map

Collection

The Collection interface (java.util.Collection) is one of the root interfaces of the Java collection classes. Though you do not instantiate a Collection directly, but rather a subtype of Collection, you may often treat these subtypes uniformly as a Collection

Java does not come with a usable implementation of the Collection interface, so you will have to use one of the subtypes(List, Set, Map and Tree.

The Collection interface just defines a set of methods (behaviour) that each of these Collection subtypes share. This makes it possible ignore what specific type of Collection you are using, and just treat it as a Collection.

This is standard inheritance, so there is nothing magical about, but it can still be a nice feature from time to time. Later sections in this text will describe the most used of these common operations.

List

The java.util.List interface is a subtype of the java.util.Collection interface. It represents an ordered list of objects, meaning you can access the elements of a List in a specific order, and by an index too. You can also add the same element more than once to a List.

Being a Collection subtype all methods in the Collection interface are also available in the List interface.

Since List is an interface you need to instantiate a concrete implementation of the interface in order to use it. You can choose between the following List implementations in the Java Collections API:

- java.util.ArrayList
- java.util.LinkedList
- java.util.Vector
- java.util.Stack

Below is the instatiation of a List

```
List listA = new ArrayList();
List listB = new LinkedList();
```

```
List listC = new Vector();
List listD = new Stack();
```

To add elements to a List you call its add() method. This method is inherited from the Collection interface. Here are a few examples:

```
List listA = new ArrayList();

listA.add("element 1");
listA.add("element 2");
listA.add("element 3");

listA.add(0, "element 0");
```

The first three add() calls add a String instance to the end of the list. The last add() call adds a String at index 0, meaning at the beginning of the list.

The order in which the elements are added to the List is stored, so you can access the elements in the same order.

You can remove elements in two ways:

- remove(Object element)
- remove(int index)

remove(Object element) removes that element in the list, if it is present. All subsequent elements in the list are then moved up in the list. Their index thus decreases by 1.

remove(int index) removes the element at the given index. All subsequent elements in the list are then moved up in the list. Their index thus decreases by 1.

Set

The java.util.Set interface is a subtype of the java.util.Collection interface. It represents set of objects, meaning each element can only exists once in a Set.

Being a Collection subtype all methods in the Collection interface are also available in the Set interface.

Since Set is an interface you need to instantiate a concrete implementation of the interface in order to use it. You can choose between the following Set implementations in the Java Collections API:

- java.util.EnumSet
- java.util.HashSet
- java.util.LinkedHashSet
- java.util.TreeSet

Here are a few examples of how to create a Set instance:

```
Set setA = new EnumSet();
Set setB = new HashSet();
Set setC = new LinkedHashSet();
Set setD = new TreeSet();
```

To add elements to a Set you call its add() method. This method is inherited from the Collection interface. Here are a few examples:

```
Set setA = new HashSet();

setA.add("element 1");
setA.add("element 2");
setA.add("element 3");
```

You remove elements by calling the remove(Object o) method. There is no way to remove an object based on index in a Set, since the order of the elements depends on the Set implementation.

Map

The java.util.Map interface represents a mapping between a key and a value. The Map interface is not a subtype of the Collection interface. Therefore it behaves a bit different from the rest of the collection types.

Since Map is an interface you need to instantiate a concrete implementation of the interface in order to use it. You can choose between the following Map implementations in the Java Collections API:

- java.util.HashMap
- java.util.Hashtable
- java.util.EnumMap
- java.util.IdentityHashMap
- java.util.LinkedHashMap
- java.util.Properties
- java.util.TreeMap
- java.util.WeakHashMap

Here are a few examples of how to create a Map instance:

```
Map mapA = new HashMap();
Map mapB = new TreeMap();
Map mapC = new Hashtable();
Map mapD = new EnumMap();
Map mapE = new IdentityHashMap();
Map mapF = new Properties();
Map mapF = new LinkedHashMap();
Map mapH = new WeakHashMap();
```

To add elements to a Map you call its put() method. Here are a few examples:

```
Map mapA = new HashMap();

mapA.put("key1", "element 1");
mapA.put("key2", "element 2");
mapA.put("key3", "element 3");
```

The three put() calls maps a string value to a string key. You can then obtain the value using the key. To do that you use the get() method like this:

```
String element1 = (String) mapA.get("key1");
```

You remove elements by calling the remove(Object key) method. You thus remove the (key, value) pair matching the key.

Generic Collections

It is possible to specify the various Collection and Map types and subtypes in the Java collection API.

The Collection interface can be specified like this:

```
Collection<Dog> dogCollection = new HashSet<Dog>();
```

This dogCollection can now only contain Dog instances. If you try to add anything else, or cast the elements in the collection to any other type than Dog, the compiler will complain.

Coding to an Interface

To Demostrate how you code to an Interface, we will look at a simple program called Calculator that is coding to a concrete class. Here is the Simple Calculator concrete class

```
public class Calculator {

    int add(int a, int b) {
        return a+b;
    }
}
```

We can run this using a Test case below

```
// Other parts Omitted for clarity
    @Test
     public void add() {
     Calculator calc = new Calculator();
     int result = calc.add(10,20);
     Assert.assertEquals("Add 2+3", 30, result);
     }
```

This calc object depends on the concrete class of type Calculator and f we we make changes to the Class, i.e swapping out a different implementation, we will have to make changes to the calc object as well. So this implementation is rigid and not flexible to allow the client and the calculator to evolve independently

Let us introduce the interface o that we seperate e implementation from the contract

```
public interface CalculatorInterface {
    int add(int a, int b);
}
```

With the Interface Done we can now create the implementation of this contract

```
public class CalculatorServiceImpl implements CalculatorInterface{
    @Override
    public int add(int a, int b) {
        return a+b;
    }
}
```

With the Implemenation done, we can now update our test method to depend on an Interface type

```
// Other parts Omitted for clarity
    @Test
     public void add() {
     CalculatorInterface calc = new CalculatorServiceImpl();
     int result = calc.add(10,20);
     Assert.assertEquals("Add 2+3", 30, result);
     }
```

This is a much improved version over the first cut. It still has drawbacks because the implemtation is still leaking into the test client. To solve this problem, we need to introduce the Springframework IoC or Dependency injection Container.

The first thing we need to do is add the SpringLibrary to our MAven POM as shown below

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>calculator</groupId>
    <artifactId>calculator</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <name>calculator</name>
    <url>http://maven.apache.org</url>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>

    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.12</version>
    </dependency>

 <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>4.1.4.RELEASE</version>
</dependency>

    </dependencies>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework</groupId>
                <artifactId>spring-framework-bom</artifactId>
                    <version>4.1.4.RELEASE</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
</project>
```

Next up is we need to create a configration package where we will put files to help us wire our dependencies. It is in this file where we shall be wirig our dependencies

```
@Configuration
public class AppConfig {
    @Bean(name="calc")
    public CalculatorInterface getService(){
        return new CalculatorServiceImpl();
    }

}
```

With the Config in place, we can now update our test class to the code below

```
public class CalculatorTest {
    private CalculatorInterface calc;

    @BeforeMethod
    public void setUp() throws Exception {
        ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);
        calc = (CalculatorInterface)ctx.getBean("calc");
```

```
    }

    @AfterMethod
    public void tearDown() throws Exception {

    }
    @Test
    public void testAdd() throws Exception {
        int result = calc.add(5,5);
        Assert.assertEquals(result,10," Sum of two numbers 5 +5 is 10");
    }
```

As you can see above, our test client is now completely decoupled from the implementation class CalculatorServiceImpl. With this in place, we can now swap out the implementation without having to change the any code on the client side.

Chapter Exercises

1. Create an Application that will make use of Collection, List, Set and Map
2. Create an application that makes use of the coding to the Interface without the use of the Springramework
3. Refactor your application in (2) to add the Springframework and add multiple implemetation of your interfaces

Please not that all Exercises should use Test Driven Development Principles. For every Feature, there has to be a Test for it.